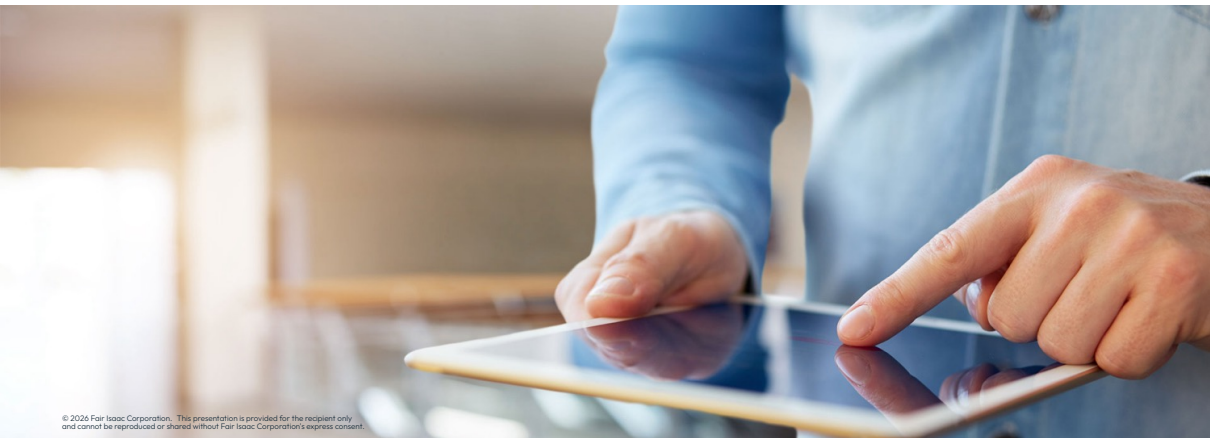


Infeasibility handling



Infeasibility handling

Infeasibility

- One of the most common difficulties is what to do when a model is infeasible...
 - ... is the model wrong?
 - ... is the data wrong?
 - ... is the problem genuinely infeasible?
 - ... where is the infeasibility so I can correct or investigate it?
- Two types of infeasibility in MIP problems:
 - **Continuous infeasibility**: LP relaxation of the MIP problem is in itself infeasible
 - **Integer infeasibility**: LP relaxation is feasible but the problem is MIP infeasible
 - These are harder to detect and handle...

Detecting infeasibility with FICO® Xpress

- Infeasibility status can be identified by the *solution status* attribute after a solve:
 - The value of `solstatus` is `xp.SolStatus.INFEASIBLE` for an infeasible problem
 - It is returned by the `p.optimize()` function, together with the *solve status*:

```
solvestatus, solstatus = p.optimize()

if solstatus == xp.SolStatus.INFEASIBLE:
    print("Problem is infeasible.")
```

- Alternatively, use the problem attribute named `solstatus`:

```
p.optimize()

if p.attributes.solstatus == xp.SolStatus.INFEASIBLE:
    print("Problem is infeasible.")
```

Detecting infeasibility with FICO® Xpress

- Diagnosis in presolve:
 - Possible to “trace” back the implications that determined an inconsistency and identify a cause
 - Solver log information about infeasibility by default:
 - The problem is infeasible due to row R67
 - Set `p.controls.trace = 1` to display a more detailed infeasibility diagnosis during presolve before calling `p.lpOptimize()`

Detecting infeasibility with FICO® Xpress

- Diagnosis in presolve:
 - Possible to “trace” back the implications that determined an inconsistency and identify a cause
 - Solver log information about infeasibility by default:
 - The problem is infeasible due to row R67
 - Set `p.controls.trace = 1` to display a more detailed infeasibility diagnosis during presolve before calling `p.lpOptimize()`
- Diagnosis using primal “phase 1” simplex solution:
 - Set `p.controls.presolve = -1` to force presolve to continue to simplex even when an infeasibility is discovered
 - The sum of infeasibilities is minimized in the solution
 - The resulting set of violated constraints and violated variable bounds provides a clear picture of what aspect of the model is causing the infeasibility



Finding help: Check the *reference manual page* for more information about infeasibility handling, and see the *diagnose_infeasible* example notebook on GitHub

Irreducible Infeasible Sets (IIS)

- Subsets of constraints, bounds and integrality conditions that form an infeasibility:
 - A model may have several infeasibilities. Repairing a single IIS may not make the model feasible
 - Works with [any problem type](#), but does not support user functions
 - Calculating IIS can be computationally expensive, so they can be [generated iteratively](#)
 - Generation of IISs available as API functions: [problem.firstIIS\(\)](#), [problem.nextIIS\(\)](#), [problem.IISAll\(\)](#), [problem.getIISData\(\)](#), [problem.writeIIS\(\)](#), ...
- Basic method for LP infeasibility:
 - Filtering after first phase primal simplex
 - Uses duals of rows and reduced costs of columns to identify subproblems containing all infeasibilities

Irreducible Infeasible Sets (IIS)

- Subsets of constraints, bounds and integrality conditions that form an infeasibility:
 - A model may have several infeasibilities. Repairing a single IIS may not make the model feasible
 - Works with [any problem type](#), but does not support user functions
 - Calculating IIS can be computationally expensive, so they can be [generated iteratively](#)
 - Generation of IISs available as API functions: [problem.firstIIS\(\)](#), [problem.nextIIS\(\)](#), [problem.IISAll\(\)](#), [problem.getIISData\(\)](#), [problem.writeIIS\(\)](#), ...
- Basic method for LP infeasibility:
 - Filtering after first phase primal simplex
 - Uses duals of rows and reduced costs of columns to identify subproblems containing all infeasibilities
- Basic method for MIP infeasibility when LP-feasible:
 1. Drop one or more constraints/bounds/integrality conditions of the problem
 - If the resulting problem becomes feasible, keep them
 2. Stop with an irreducible set when nothing can be removed

Irreducible Infeasible Sets (IIS)

- Generate a single IIS with `p.firstIIS()` or `p.nextIIS()`, and export it to a file using `p.writeIIS()`:

```
if p.firstIIS(1) == 0:  # returns 0 if successful
    p.writeIIS(iis=1, filename="iis", filetype=0, flags="l")  # export in LP format
```

- This step might likely be enough to diagnose an infeasible problem

Irreducible Infeasible Sets (IIS)

- Generate a single IIS with `p.firstIIS()` or `p.nextIIS()`, and export it to a file using `p.writeIIS()`:

```
if p.firstIIS(1) == 0: # returns 0 if successful  
    p.writeIIS(iis=1, filename="iis", filetype=0, flags="l") # export in LP format
```

- This step might likely be enough to diagnose an infeasible problem
- Alternatively, use `p.IISAll()` to generate all IISs (only for LPs):
 - The `numiis` attribute to retrieve the number of IISs found so far

```
solvestatus, solstatus = p.optimize()
```

```
if solstatus.name == "INFEASIBLE":  
    p.IISAll() # generate all IIS  
    numiis = p.attributes.numiis # retrieve the number of IISs generated so far  
    print(f"The problem has {numiis} IISs")
```

Irreducible Infeasible Sets (IIS)

- Use `p.getIISData()` to retrieve information for a specific IIS, after `p.IISAll()`, such as:
 - Size, variables (row and column vectors) and conflicting sides of the variables, duals, reduced costs, and isolation rows and columns, if available

```
for i in range(1,numiis+1):
    miisrow, miiscol, constrainttype, colbndtype, duals, rdcs, \
        isolationrows, isolationcols = [], [], [], [], [], [], [], []

    # retrieve the i'th IIS data
    p.getIISData(i,miisrow, miiscol, constrainttype, colbndtype, \
        duals, rdcs, isolationrows, isolationcols)

    print("iis data:", miisrow, miiscol, constrainttype, colbndtype, \
        duals, rdcs, isolationrows, isolationcols)
```

Irreducible Infeasible Sets (IIS)

- Perform the [isolation](#) identification procedure for an Irreducible Infeasible Set (IIS) via [problem.IISisolations\(iis\)](#):
 - An IIS isolation is a special constraint or bound in an IIS
 - They indicate the likely cause of each independent infeasibility and [which constraints or bounds to drop or modify](#)
 - Removing/repairing constraints and bounds in an IIS isolation will remove all infeasibilities in the IIS without increasing the infeasibilities in other IISs
 - This procedure is computationally expensive, and is done via [p.IISisolations](#) for an identified IIS:

```
p.IISAll()  
numiis = p.attributes.numiis  
for i in range(1,numiis+1):  
    p.IISisolations(i)
```

Infeasibility repair utility

- A **two-phase approach** to solve the problem with a minimum weighted sum of violations of relaxed constraints and bounds:
 - A set of **preferred constraints and bounds** is indicated, and the Optimizer attempts to identify a 'solution' that violates them minimally while satisfying all other constraints and bounds
- Available via the **p.repairInfeas()** method:
 - Phase I:
 - Adds *penalty* variables to constraints and bounds
 - Minimizes the weighted sum of penalties to obtain a minimally weighted infeasible solution

Infeasibility repair utility

- A **two-phase approach** to solve the problem with a minimum weighted sum of violations of relaxed constraints and bounds:
 - A set of **preferred constraints and bounds** is indicated, and the Optimizer attempts to identify a 'solution' that violates them minimally while satisfying all other constraints and bounds
- Available via the **p.repairInfeas()** method:
 - Phase I:
 - Adds *penalty* variables to constraints and bounds
 - Minimizes the weighted sum of penalties to obtain a minimally weighted infeasible solution
 - Phase II:
 - Adds a constraint to restrict the sum of violations to the Phase I solution (plus a tolerance)
 - Solves for the original objective
- More advanced functions are available to allow weights to be specified for each constraint and bound: **p.repairWeightedInfeas()**, **p.repairWeightedInfeasBounds()**



Finding help: Check the *reference manual page* for more information on the *repairInfeas** functions

Infeasibility repair utility

- Example:

```
p.optimize()
```

```
if p.attributes.solstatus.name == "INFEASIBLE":
```

```
    penalty = 'c' # each penalty is the reciprocal of the corresponding preference  
    phase2 = 'o'  # use the objective sense of the original problem  
    flags = 'g'   # do the tree search  
    delta = 0     # the relaxation multiplier in phase II
```

```
# integer arguments represent the preferences "lepref, gepref, lbpref, ubpref"  
p.repairInfeas(penalty, phase2, flags, 1, 0, 0, 0, delta)
```

- A preference value of 0 results in the row or bound not being relaxed, and a negative preference indicates that a quadratic penalty cost should be applied
- The sum of violations in phase II is restricted to be no greater than $(1 + \text{delta}) * p$ (with p = sum of violations in phase I)

Infeasibility repair utility

- `p.repairWeightedInfeas()` allows per-row and per-column preferences, giving precise control over which constraints can be relaxed:

```
n_rows = p.attributes.rows
n_cols = p.attributes.cols
row_types = p.getRowType(0, n_rows - 1)

# protect equality constraints (E) from relaxation, allow relaxation of others
le_prefs = [0.0 if t == 'E' else 1.0 for t in row_types]
ge_prefs = [0.0 if t == 'E' else 1.0 for t in row_types]
lb_prefs = [0.0] * n_cols # do not relax lower bounds
ub_prefs = [0.0] * n_cols # a choice: upper bounds could also be relaxed

p.repairWeightedInfeas(lepref=le_prefs, gepref=ge_prefs,
                       lbpref=lb_prefs, ubpref=ub_prefs)
```


Infeasibility repair utility

- Preference values for `repairWeightedInfeas()`:
 - 0: constraint is **never relaxed**
 - **positive value**: constraint may be relaxed; higher values make relaxation cheaper (weight = $1/\text{preference}$)
- $\Rightarrow$ Less programming effort than manual IIS analysis, but less fine-grained and likely to give a different solution than manual constraint tightening

Avoiding infeasibility

- Try to introduce some flexibility into the problem by adding slack variables:
 - Incorporate explicit **slack/surplus variables** in constraints, so that if demand cannot be met or more resources are required, this is apparent from a feasible solution
 - Rather than making the model infeasible, the slack variables act as **pressure valves**
 - For example, replace:

$$\sum_f ship_{fc} \geq DEM_c \forall c$$

by

$$\sum_f ship_{fc} \geq DEM_c - xdem_c \forall c$$

- Penalize the extra variables in the objective function with a high enough cost so that they are at **zero unless the model is infeasible**:
 - If no actual penalty costs (e.g. price for buying additional quantities) are known, **coefficients should be one order of magnitude larger** than any other coefficient...
 - ... but **within the recommended numerical tolerances**!

Avoiding infeasibility - Example

- **Objects:**

- suppliers, depots, customers

- **Data:**

- availability $AVAIL_s$, demand DEM_c , transportation cost per route: $COST_{sd}$ and $COST_{dc}$

- **Decisions:**

- quantities to be shipped: $flow_{sd}$ and $flow_{dc}$
- slack variables: x_{avail}_s , x_{demand}_c

- **Constraints:**

- remain within supply limits: $\forall s : \sum_d flow_{sd} \leq AVAIL_s + x_{avail}_s$
- satisfy customer demand: $\forall c : \sum_d flow_{dc} \geq DEM_c - x_{demand}_c$
- flow balance in depots: $\forall d : \sum_s flow_{sd} = \sum_c flow_{dc}$

- **Objective:**

- minimize total cost of selected routes and penalize slack: $\min \sum_s \sum_d COST_{sd} \cdot flow_{sd} + \sum_d \sum_c COST_{dc} \cdot flow_{dc} + \sum_s PENALTY \cdot x_{avail}_s + \sum_c PENALTY \cdot x_{demand}_c$

Avoiding infeasibility

- Important to keep in mind when adding slack/surplus variables:
 - Always assert the values of the added variables to ensure they are equal to zero (feasible problem)
 - If the slack variables are active (non-zero), it means that the original problem is infeasible, and you should go back and find out why
 - Be careful not to set penalty costs so large as to cause numerical issues
 - Be aware not to treat an infeasible solution as a feasible solution!
- Putting slack variables in all constraints **can slow down the optimization**:
 - An **alternative approach** is to solve the problem first without slack variables, then, if the problem is infeasible, add the slack variables to the formulation and re-solve
 - Another alternative could be to use multi-objective optimization to first find a minimally violated solution and then solve for the intended objective



Note: *Slack/surplus variables should **only be introduced in inequality constraints** to ensure that the model remains valid!*

Python Notebook [PN-3]: Infeasibility handling in Portfolio Optimization

- Navigate to [Exercise 3 - Infeasibility handling](#) in the notebook
- The portfolio model is [intentionally infeasible](#): sector allocation limits and regional minimum requirements conflict
- The notebook walks through the full diagnostic workflow:
 1. Solve and detect infeasibility via [p.attributes.solstatus](#)
 2. Call [diagnose_infeasible\(\)](#) helper to compute IIS and isolations
 3. Interpret the output: which constraint type is causing the infeasibility?
 4. [Manual fix](#): relax the binding sector constraints and re-solve
 5. [Automatic repair](#): apply [p.repairWeightedInfeas\(\)](#) with equality constraints protected
- Both approaches yield different portfolio allocations - visualized side by side



Thank You

www.fico.com/optimization